



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

JPA PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on entity lifecycle, JPQL, relationships, and Spring Data

TOTAL: 10 QUESTIONS

Q1 What is JPA and how does it relate to Hibernate?

Threat Model

Q6 Explain the three JPA inheritance mapping strategies.

Observability

Q2 Describe the four JPA entity lifecycle states.

Architecture

Q7 How do derived query methods work in Spring Data JPA?

Lifecycle

Q3 What does EntityManager.persist(), merge(), and remove() each do?

Configuration

Q8 What is the role of @Transactional and how does propagation work?

Compliance

Q4 How do you write a JPQL query and what makes it portable?

Hardening

Q9 How do you map a composite primary key in JPA?

Testing

Q5 What is the difference between @Entity, @MappedSuperclass, and @Embeddable?

SSO / IdP

Q10 What is a JPQL DTO projection and why use it instead of returning entities?

Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 JPA (Jakarta Persistence API) is a

Mitigation

JPA (Jakarta Persistence API) is a specification defining how Java objects map to relational data. Hibernate is the most popular JPA provider that implements the spec. Code written to the JPA API can switch providers without changes.

A6 SINGLE_TABLE: all subclasses in one table,

Observability

SINGLE_TABLE: all subclasses in one table, a discriminator column identifies the type fast but nullable columns. JOINED: subclass-specific columns in child tables joined to the parent normalized. TABLE_PER_CLASS: full columns in each table no joins but no polymorphic queries.

A2 New (transient): not associated with a

Primitives

New (transient): not associated with a persistence context. Managed: changes tracked, flushed to DB on commit. Detached: context closed, changes ignored. Removed: entity marked for deletion; DELETE runs at flush.

A7 Spring Data parses the method name

Automation

Spring Data parses the method name at startup and generates JPQL. findByEmailAndActiveTrue() generates SELECT u FROM User u WHERE u.email = ?1 AND u.active = true. Supported keywords include Is, Not, Between, LessThan, OrderBy, etc.

A3 persist() makes a new entity managed

Hardening

persist() makes a new entity managed (INSERT on flush). merge() copies state of a detached entity onto a managed instance and returns it. remove() marks a managed entity for deletion. None issues SQL immediately that happens at flush/commit.

A8 REQUIRED (default) joins an existing

Governance

REQUIRED (default) joins an existing transaction or starts one. REQUIRES_NEW always starts a new transaction, suspending the outer one. MANDATORY throws if there is no active transaction. The proxy intercepts method calls to manage commit/rollback.

A4 JPQL uses entity class names and

Gotchas

JPQL uses entity class names and field names instead of table/column names. Example: SELECT u FROM User u WHERE u.email = :email. The JPA provider translates to the target database dialect, making queries portable across databases.

A9 With @EmbeddedId: create a @Embeddable

Assessment

With @EmbeddedId: create a @Embeddable class holding the key fields and annotate the entity field with @EmbeddedId. With @IdClass: annotate the entity with @IdClass(MyKey.class) and repeat the key fields annotated @Id in the entity itself.

A5 @Entity maps to its own table.

Federation

@Entity maps to its own table. @MappedSuperclass shares mapped fields with subclass entities but has no table of its own. @Embeddable defines a value object whose columns are embedded in the owning entity's table.

A10 A constructor expression (SELECT new

Optimization

A constructor expression (SELECT new com.app.dto.UserDto(u.id, u.name) FROM User u) returns lightweight DTOs instead of full managed entities. This avoids loading unused columns, prevents accidental lazy-loading, and is ideal for read-only API responses.